CALIFORNIA STATE UNIVERSITY
LONG BEACH

CECS 341: Computer Architecture & Organization

Hossein Sayadi, PhD

Department of Computer Engineering & Computer Science
California State University, Long Beach

Fall 2025

Lecture Outline

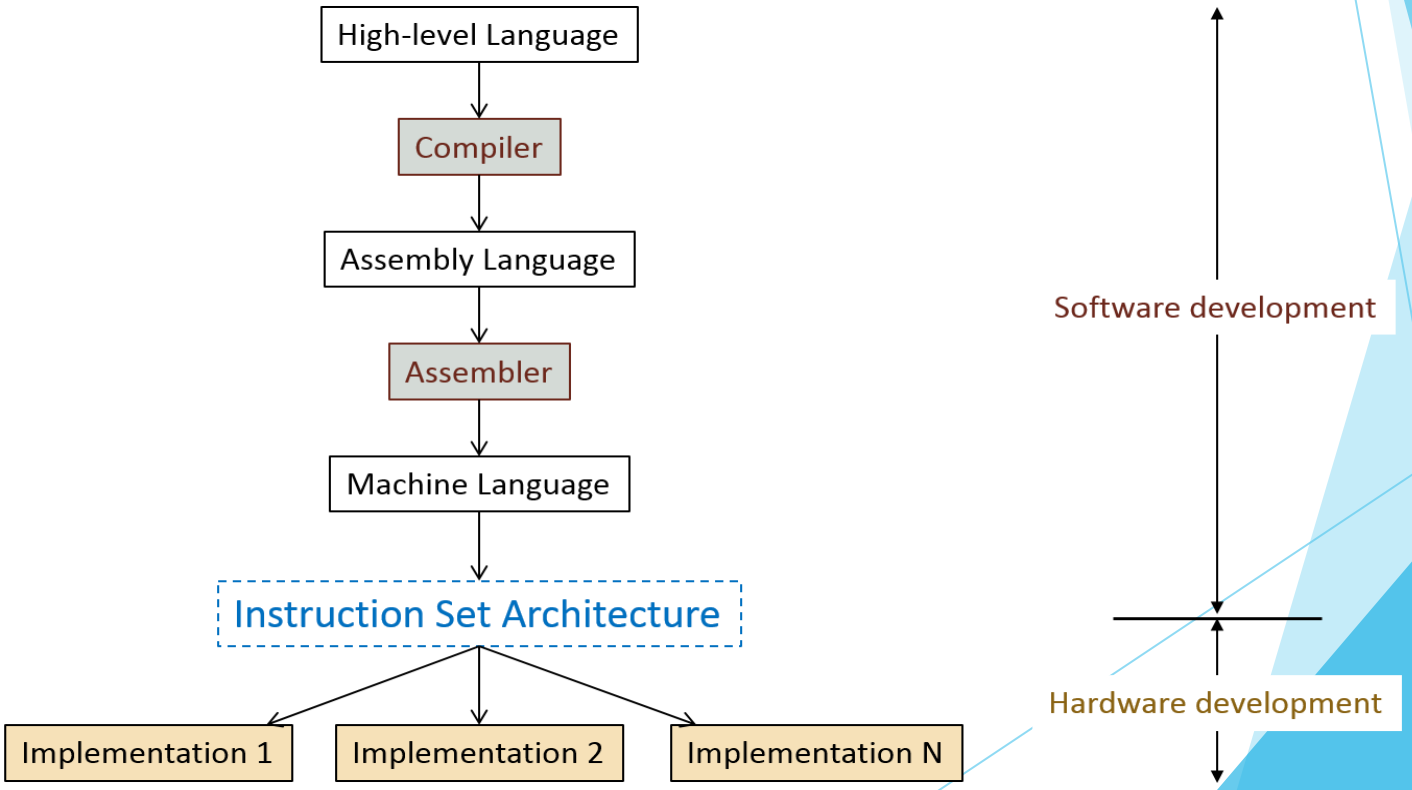
- Computer Architecture and Organization
- Different Computer Architectures
  - Von Neumann (Princeton)
  - Harvard
  - Modified Harvard
  - RISC, CISC
- What is an Instruction Set Architecture?
  - Definition and description
  - Attributes of a processor described by the ISA.

Introduction

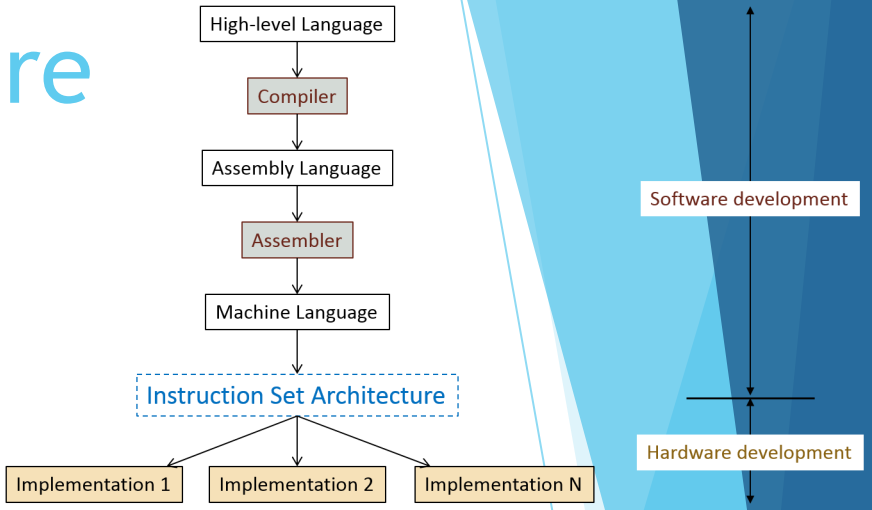
Architecture and Organization

Architecture and Organization

- The relationship between computer *architecture*, hardware *organizations*, and software development is illustrated below:

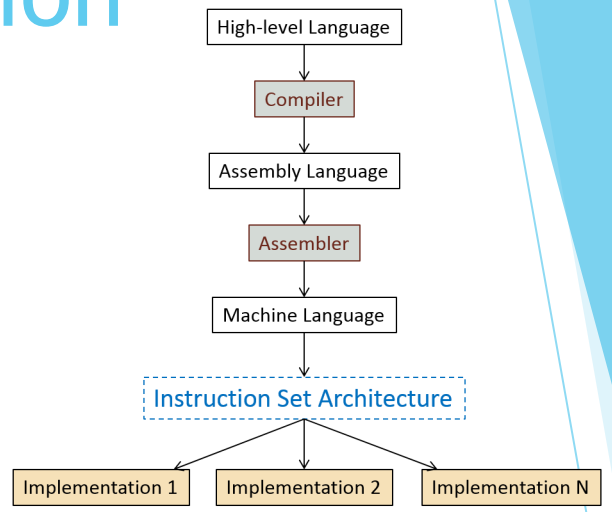


Computer Architecture



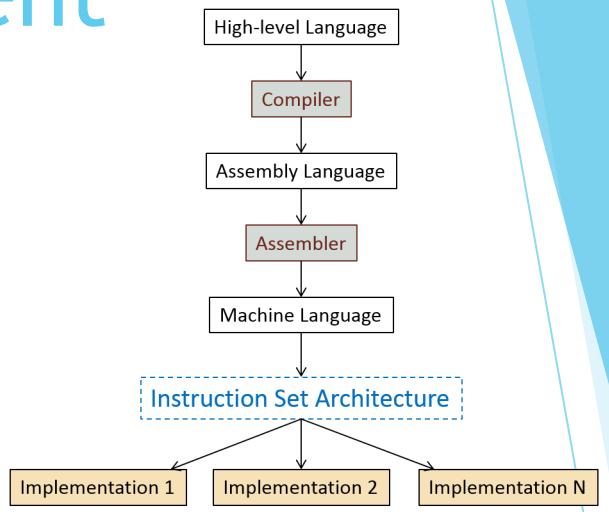
- Instruction Set Architecture (ISA).
- Specifies **what** the processor can do, not **how** it is implemented.
- Defines the capabilities of the processor.
- Defines the processor's *programming model*.
- Is the ***programmer's view*** of the processor.
- Serves as the *interface* between the processor hardware and the low-level instructions executed by the processor.
- Allows hardware and software to be developed independently.

Computer Organization



- Hardware Implementation.
- Specifies **how** the processor architecture is realized.
- Specifies the technology used, the method of interconnecting the datapath components, the width of the internal buses, etc.
- Many implementations can be realized for the same processor architecture.
- Results in a *family* of processors, each with a different price/performance specification.

Compiler Development



- Software Development.
- Compilers are designed for a processor architecture.
- High-level programs are compiled for a processor architecture.
- Low-level programs are written for a processor architecture.
- Low-level code is portable across *different implementations* of the *same architecture*.

Architectures

Stored Program Computer.

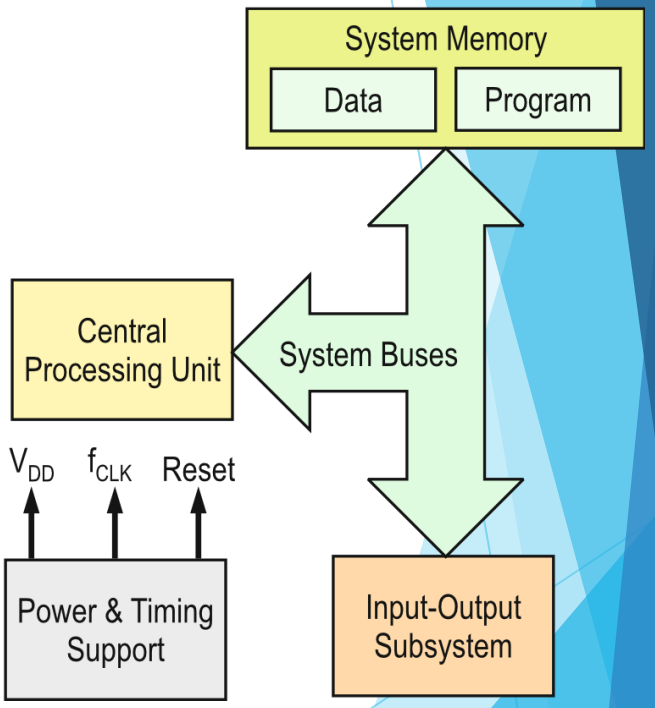
Von Neuman (Princeton), Harvard.

RISC, CISC.

Examples: x86, MC68000, ARM, MIPS.

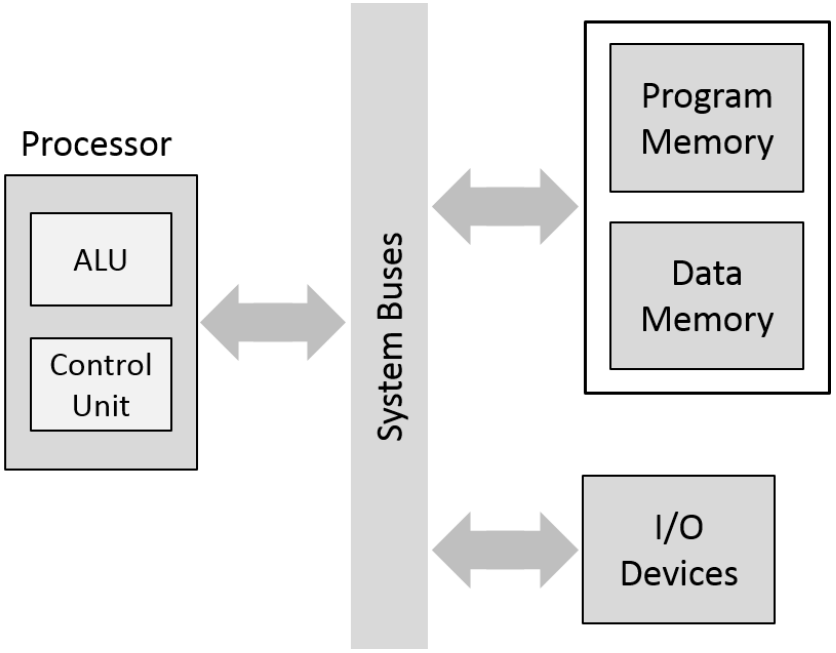
Stored Program Computer

- A computer in which both program instructions and data are stored in electronic memory.
  - Can be in the same memory or in separate memories.
- Instructions and data are stored in memory in exactly the same way.
  - Stored as a series of 0's and 1's.
  - Accessed in the same amount of time.
  - The only difference between instructions and data is how they are interpreted by the processor.



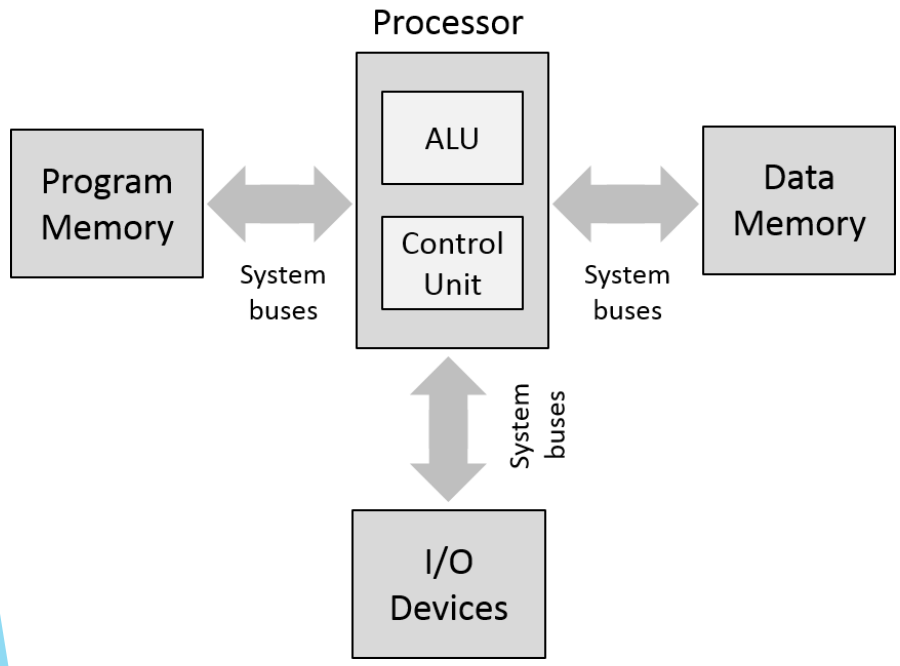
"Introduction to Embedded Systems using Microcontrollers and the MSP430"; Jimenez, Palomera, Couvertier; p. 82

Von Neumann Architecture



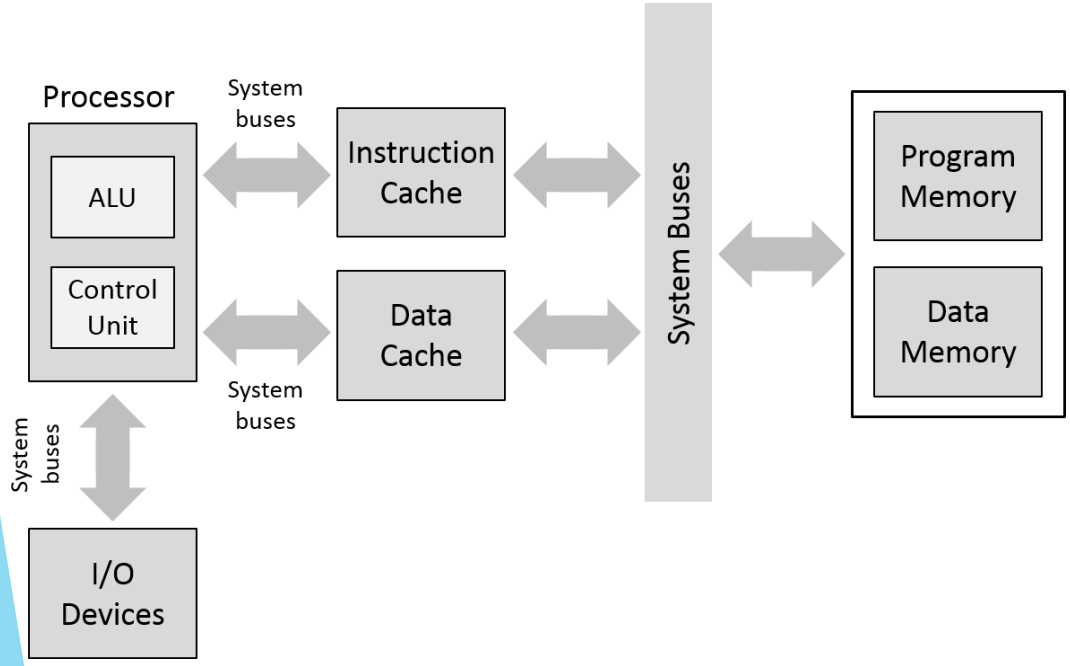
- Uses a single bus to access both instructions and data (in memory).
- The processor can either fetch an instruction or read/write data.
  - Cannot do both at the same time.
- The resulting system requires less complex hardware, and uses a single, sequential memory.
- Von Neumann Bottleneck
  - Processor performance reduced because program and data memory cannot be accessed at the same time.

Harvard Architecture



- Includes two separate buses, one to access instruction memory and the other to access data memory.
- The processor can access both memories at the same time.
  - Can fetch an instruction and read/write data at the same time.
  - Eliminates *structural hazards* in pipelined processors.
- Increases the throughput of the machine by enabling it to always pre-fetch the next instruction.
- The cost of such a system is the hardware complexity.
- Commonly used in DSPs.

Modified Harvard Architecture



- Includes a single main memory for both instructions and data.
- Has separate caches
  - One for instructions
  - One for data
- Processor fetches instructions from instruction cache.
- Processor reads/writes data from/to data cache.
- On cache miss, instructions or data are transferred from main memory to the appropriate cache.

Reduced Instruction Set Computer (RISC)

- A type of processor architecture that includes a set of simple instructions that can each be executed in a single clock cycle.
- Characteristics of a RISC architecture:
  - Set of simple instructions.
  - Each instruction is executed in a single clock cycle.
  - Uses specialized instructions (load and store) to access memory.
  - Uniform instruction format (requires less decoding).
  - Large set of identical general purpose registers (simplifies compiler design).
  - Simple addressing modes.
  - Can be easily *pipelined* (a technique that allows multiple instructions to be executed simultaneously).

First RISC projects:
- IBM 801
- Stanford MIPS
- Berkeley RISC

RISC families:
- DEC Alpha
- ARM
- Intel i860, i960
- MIPS
- PowerPC
- SPARC (RISC)

Complex Instruction Set Computer (CISC)

- A type of processor architecture that includes a set of complex instructions that can each implement several low-level operations.
- Characteristics of a CISC architecture:
  - Set of complex instructions that directly support high-level programming constructs.
  - Instructions require multiple clock cycles.
    - Number of CC's different for different instructions.
  - All instructions can access memory.
  - Different instruction formats and lengths.
    - Determined by complexity of instruction.
  - Small code size.
    - HLL program compiles into fewer instructions.

CISC architectures:
- IBM System/360
- PDP-11
- VAX
- Motorola 68000
- Intel 8080, x86
- Zilog Z80

CISC vs. RISC: Comparison

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy

Example: x86

- The x86 Architecture
  - Designer: Intel, AMD
  - Data bits: 16-bit (1978), 32-bit (1985), 64-bit (2003)
  - Design: Complex Instruction Set Computer (CISC)
  - Type: Register-Memory
  - Instruction Length: Variable (1 to 15 bytes)

- Implementations of the x86 architecture
  - 8086, 80186, 80286, 80386, 80486, Pentium, Athlon, Celeron,
  - Intel Core i3, Core i5, Core i7.

Example: ARM

- The ARM Architecture
  - Designer: (Acorn) Advanced RISC Machine
  - Data bits: 32-bit (1985), 64-bit (2011)
  - Design: Reduced Instruction Set Computer (RISC)
  - Type: Register-Register (Load-Store)
  - Instruction Length: Fixed (32 bits)
- Implementations of the ARMv7-A architecture
  - ARM Cortex-A5, ARM Cortex-A7, ARM Cortex-A8, ARM Cortex-A9, etc.
- Implementations of the ARMv8-A architecture
  - ARM Cortex-A35, ARM Cortex-A53, ARM Cortex-A57, ARM Cortex-A72.

Example: MIPS

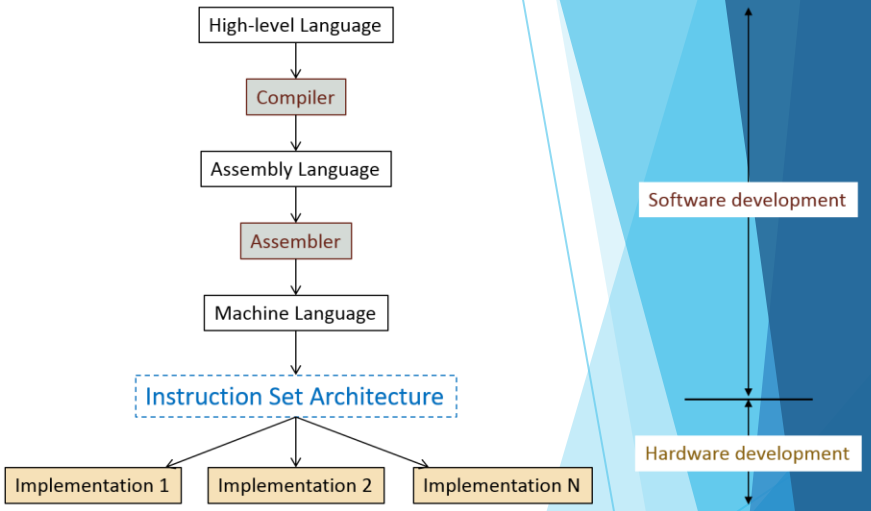
- The MIPS Architecture
  - Designer: MIPS Technologies, Inc.
  - Data bits: 32-bit (1981), 64-bit (later)
  - Design: Reduced Instruction Set Computer (RISC)
  - Type: Register-Register (Load-Store)
  - Instruction Length: Fixed (32 bits)
- Implementations of the MIPS architecture
  - R2000, R6000, R4000, R8000.

Instruction Set Architecture

Processor characteristics described by the ISA.

Instruction Set Architecture

- The ISA defines the ***behavior*** of a processor.
- It specifies the following:
  - Instruction set
  - Register set
  - Addressing modes
  - Instruction formats
  - Data width
  - Address width
  - Byte-addressable or word-addressable memory
  - Endian-ness
- The ISA differentiates one processor family from another.
  - x86, MC68000, ARM, MIPS, etc.



Instruction Set

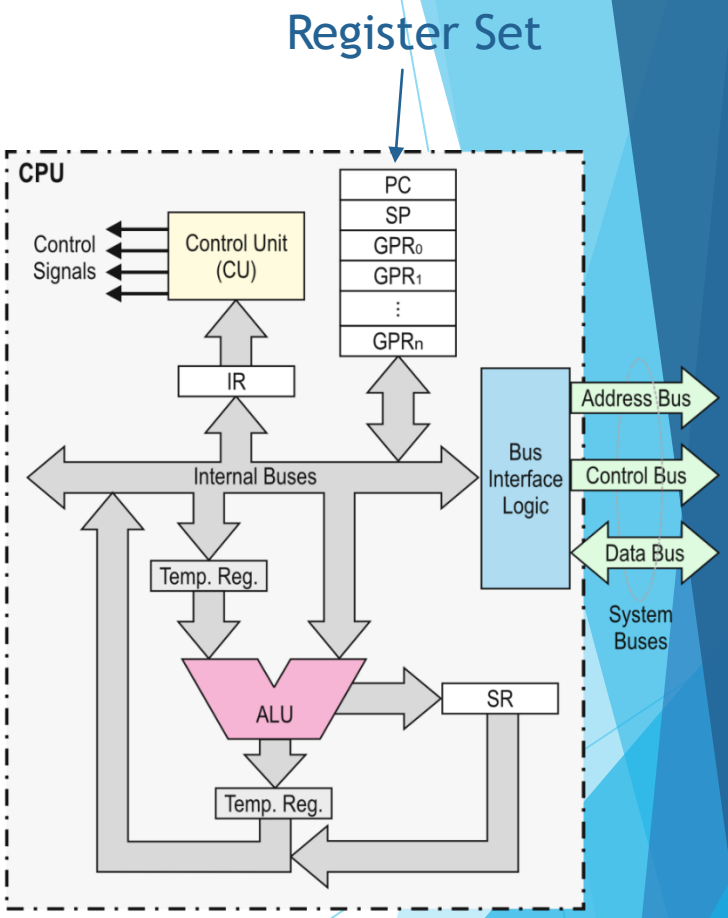
- The set of instructions that can be executed by the processor.
- Not to be confused with *high-level* language instructions.
- The instruction set is composed of *low-level* instructions.
  - Assembly language human-readable form of the instructions.
  - Machine language machine-readable form of the instructions.
- The instruction set includes all low-level instructions needed to implement any high-level language constructs.
- The instruction set is processor-specific.
  - Cannot be executed on a different processor.
  - A high-level language program must be compiled for a specific processor.
    - Resulting low-level code cannot be executed on a different processor.
  - Low-level code can be executed only on the CPU for which it was written.

Instruction Types

- Processor instructions can be categorized as follows:
  - Data processing
    - Arithmetic instructions
    - Logical instructions
  - Data transfer
    - Instructions to transfer data from memory to registers (Load)
    - Instructions to transfer data from registers to memory (Store)
  - Data movement
    - Input/output instructions
  - Control
    - Conditional branch instructions
    - Unconditional jump instructions
    - Test instructions

Register Set

- User-accessible registers.
  - General-purpose, special-purpose.
  - Accessible to assembly language programmer.
  - Store instruction operands.
- High-speed registers.
  - Fast access to instruction operands.
  - Highest level of memory hierarchy.
- Implemented using a register file.
- Located on the same silicon chip as the processor.
  - Decreases latency; increases speed.

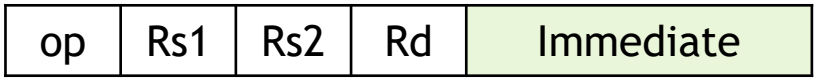


Addressing Modes

- Used to determine the location of the instruction operands.
  - Each instruction operand has an associated addressing mode.
  - Different operands in the same assembly language instruction can have different addressing modes.
- Multiple addressing modes provide flexibility.
  - Allows the location of the source and destination operands to be specified in a variety of ways.
- Instruction operands can be found
  - Within the instruction
  - In one of the general purpose registers (in the register file).
  - In (data) memory.

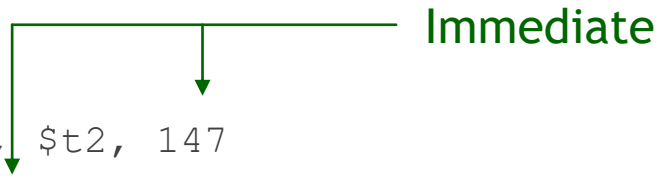
Addressing Modes

- Immediate



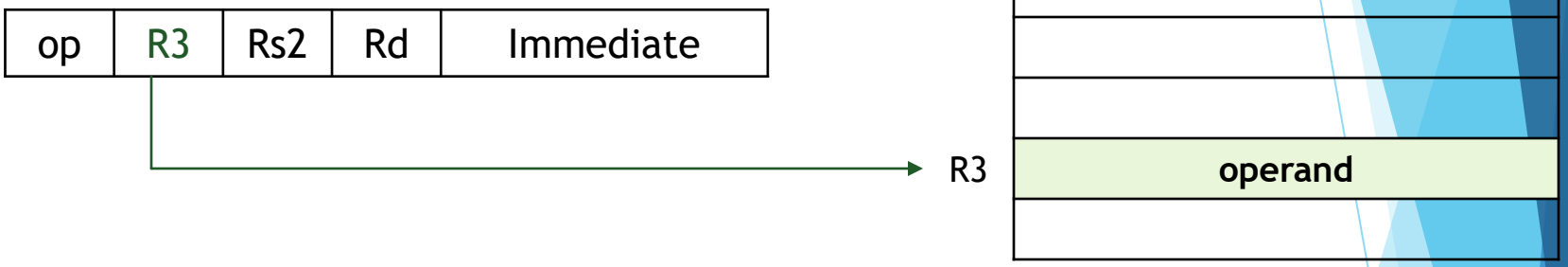
  - Operand is part of the instruction.
  - Effective Address (EA) = immediate field of the instruction.

- Examples
  - MIPS: `addi $t1, $t2, 147`
  - MIPS: `lw $s4, 28($s5)`
  - MIPS: `sw $s6, 12($t6)`



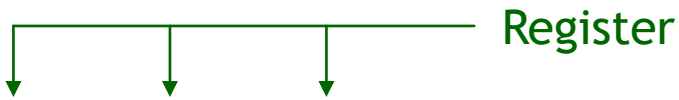
Addressing Modes

- Register



  - Operand is in the specified register.
  - Effective Address (EA) = Register number = Rn.

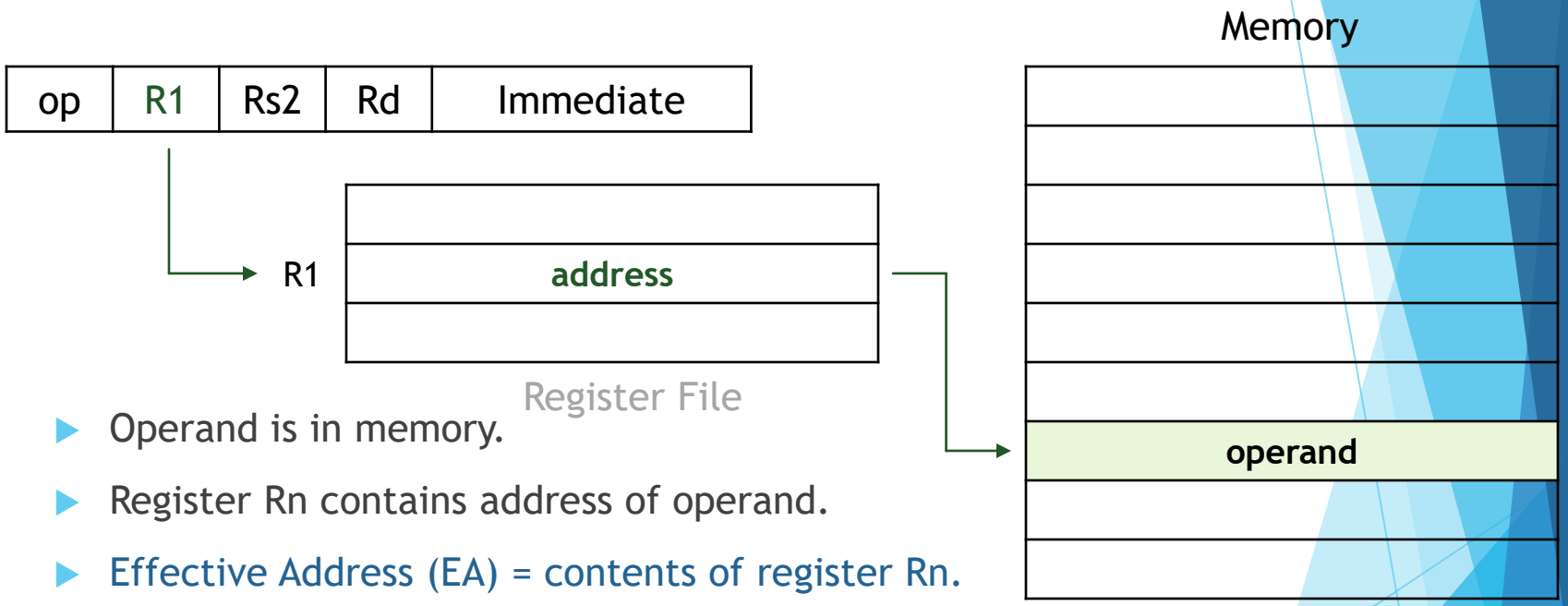
- Examples



  - MIPS: `add $s2, $s1, $s0`

Addressing Modes

- Register Indirect

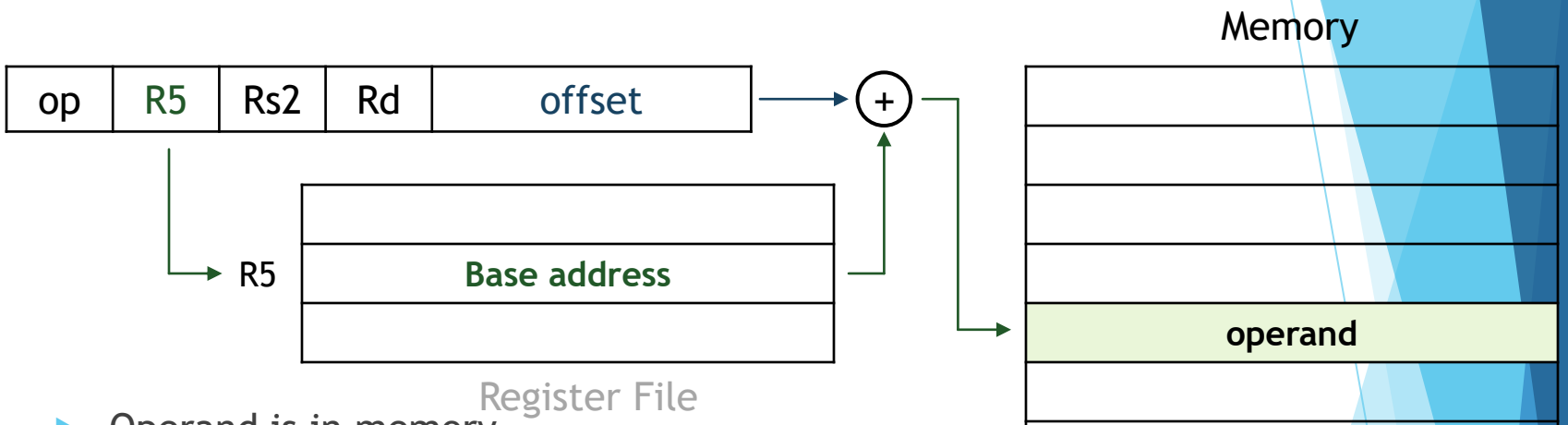


  - Operand is in memory.
  - Register Rn contains address of operand.
  - Effective Address (EA) = contents of register Rn.
- Examples
  - MIPS: `lw $s3, 0($t4)`

Register containing address of operand

Addressing Modes

- Indexed (aka. Base + Offset)



  - Operand is in memory.
  - Register Rn contains base address; Immediate field is offset.
  - Effective Address (EA) = contents of register Rn + offset.
- Examples
  - MIPS: `sw $t1, 12($s0)`

address of operand

Addressing Modes

Addressing Mode	Description
Register	All operands in registers.
Immediate	Constant (immediate) value included in instruction.
Absolute	Address specified in instruction.
Register Indirect	Address of operand is specified in a register; register number specified in instruction; EA = R[Rn]
Indexed (aka. Base + Offset)	Constant (offset) value included in instruction; base register specified in instruction; EA = R[base] + offset
Base + Index	Base register specified in instruction; index register specified in instruction; EA = R[base] + R[index]
Base + Index + Offset	EA = R[base] + R[index] + offset
PC-relative	Constant (offset) value specified in instruction; base register is PC; EA = R[PC] + offset

EA = Effective Address; address used to access (read or write) memory.
R[Rn] = contents of register Rn

Instruction Format(s)

- The bits of a machine language instruction are organized into fields which are defined by the *instruction format*.
- The fields of a machine language instruction include:
  - Opcode (specifies the operation to be performed).
  - Source operand(s).
  - Destination operand.
- The instruction (format) also specifies
  - The addressing mode of each operand (implicitly or explicitly).
  - The address of the next instruction to execute (implicitly or explicitly).
- The format is used to interpret the instruction.
- Most architectures include a set of similar instruction formats.

Instruction Length

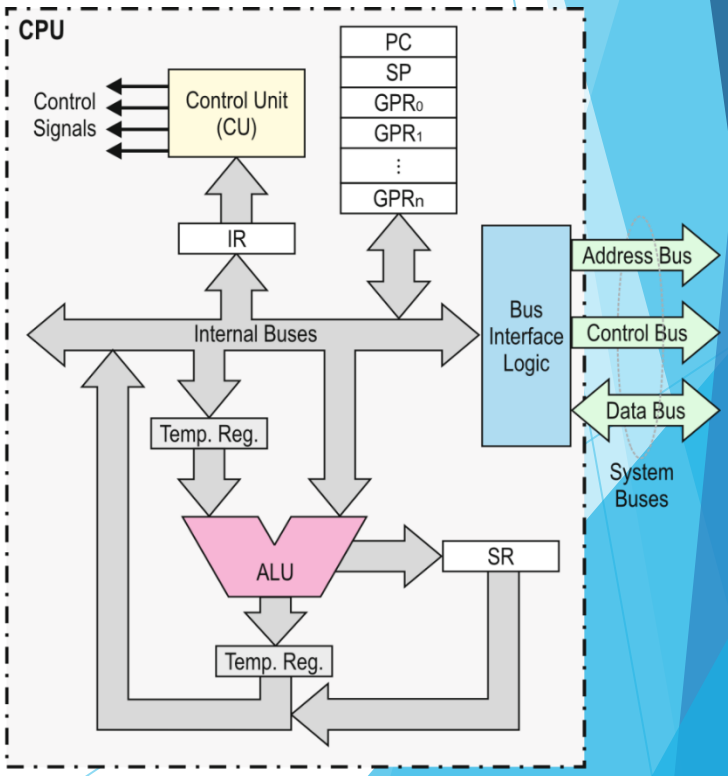
- Determines the richness and flexibility of the processor.
  - As seen by the assembly language programmer.
- *Trade-off:* powerful instruction set vs. save memory space.
- Fixed-length instructions
  - Simplifies instruction decoding.
  - Limits instruction set designer.
- Variable-length instructions
  - Increases processor complexity.
  - Provides greater flexibility for the design of the instruction set.

- Instruction length must be an integer multiple of the word length.

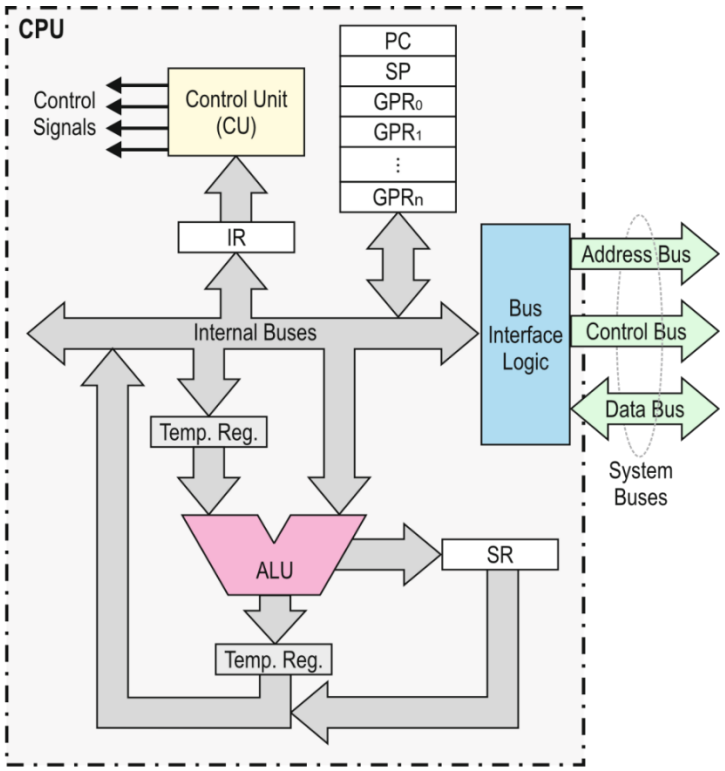
Processor Size (in bits)

- Defines the size of
  - The ALU operands.
  - The general-purpose registers (in the register file).
  - Most of the special purpose registers.
  - The internal bus.

- Example: 32-bit processor
  - 32-bit ALU operands
  - 32-bit registers
  - 32-bit internal bus

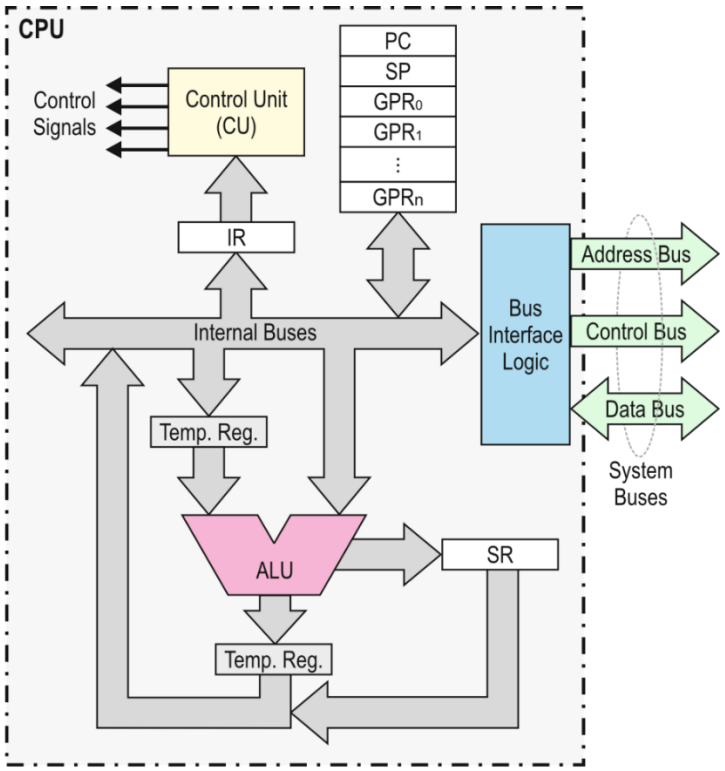


Data Bus



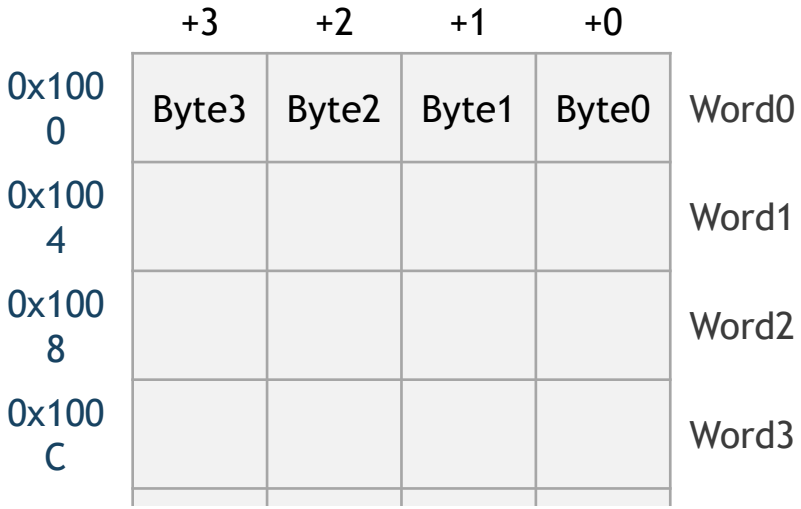
- High-level languages typically include multiple data types.
- Each data type has a specified width.
  - Character 8 bits
  - Integer 16 bits or 32 bits
  - Floating Point 32 bits or 64 bits
- The ISA should support data transfer of different sizes
  - Byte, half-word, and word.
- Smallest data *unit* is typically the **byte**.
- Largest data *unit* is the **word**.
  - Presently, a word is either 32 or 64 bits.

Address Bus

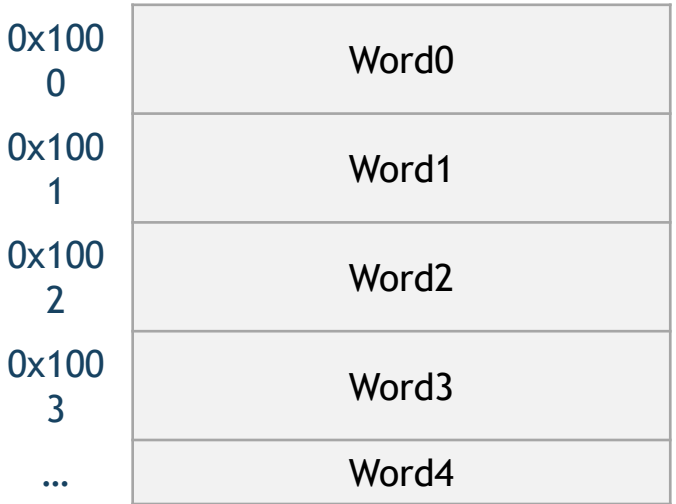


- Determines the size of the memory space that can be accessed by the processor.
- For a *byte-addressable* processor with an ***n***-bit address bus,
  - Accessible memory space = 2^n bytes
- For a *word-addressable* processor with an ***n***-bit address bus,
  - Accessible memory space = 2^n words

Byte- and Word- Addressable Memory

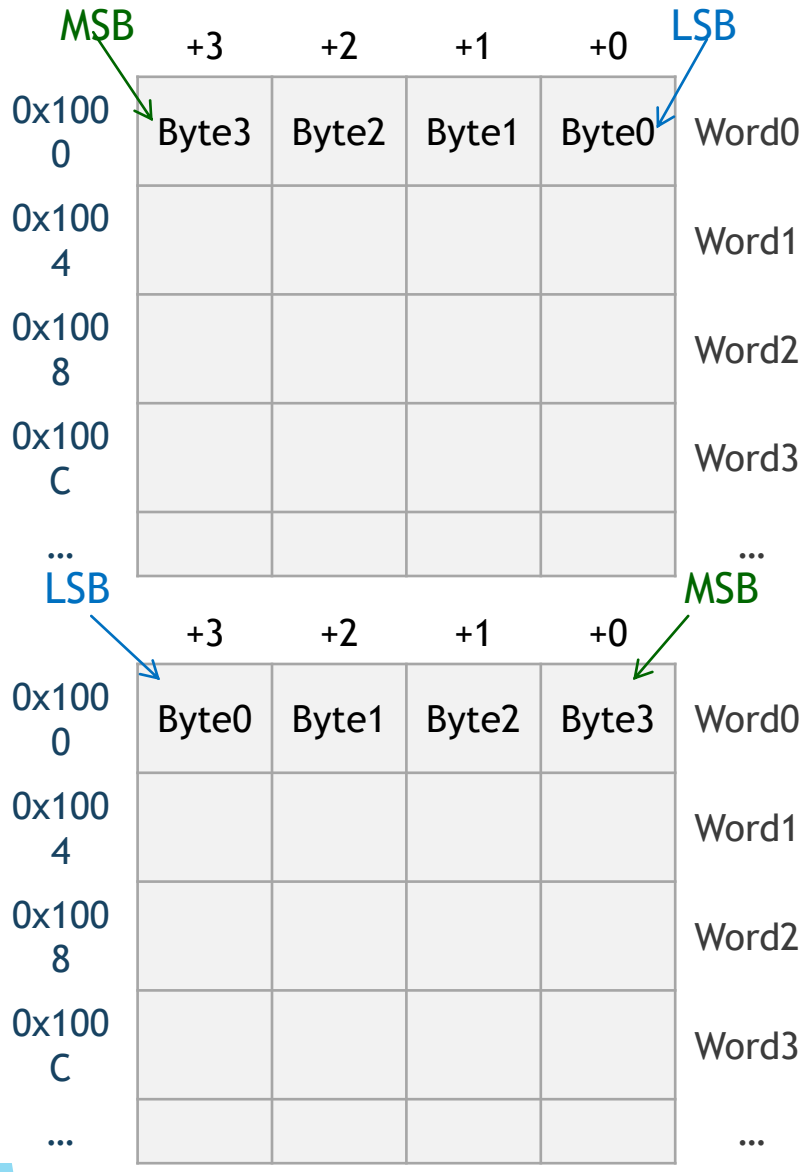


- Byte-Addressable
  - Each byte uniquely addressable.
  - Bytes located at sequential addresses.
  - Each word located at a word boundary.
    - Each word located at an address that is divisible by four (ie. # bytes/word).



- Word-Addressable
  - Bytes are not uniquely addressable.
  - Words located at sequential addresses.

Endian-ness



- Little Endian
  - Least-significant byte (LSB) of word stored at lowest address.
  - Most-significant byte (MSB) of word stored at highest address.
- Big Endian
  - Most-significant byte (MSB) of word stored at lowest address.
  - Least-significant byte (LSB) of word stored at highest address.

MIPS Instruction Set

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	Three register operands
	subtract	sub $s1,$s2,$s3	$s1 = $s2 – $s3	Three register operands
	add immediate	addi $s1,$s2,20	$s1 = $s2 + 20	Used to add constants
Data transfer	load word	lw $s1,20($s2)	$s1 = Memory[$s2 + 20]	Word from memory to register
	store word	sw $s1,20($s2)	Memory[$s2 + 20] = $s1	Word from register to memory
	load half	lh $s1,20($s2)	$s1 = Memory[$s2 + 20]	Halfword memory to register
	load half unsigned	lhu $s1,20($s2)	$s1 = Memory[$s2 + 20]	Halfword memory to register
	store half	sh $s1,20($s2)	Memory[$s2 + 20] = $s1	Halfword register to memory
	load byte	lb $s1,20($s2)	$s1 = Memory[$s2 + 20]	Byte from memory to register
	load byte unsigned	lbu $s1,20($s2)	$s1 = Memory[$s2 + 20]	Byte from memory to register
	store byte	sb $s1,20($s2)	Memory[$s2 + 20] = $s1	Byte from register to memory
	load linked word	ll $s1,20($s2)	$s1 = Memory[$s2 + 20]	Load word as 1st half of atomic swap
	store condition. word	sc $s1,20($s2)	Memory[$s2+20]=$s1;$s1=0 or 1	Store word as 2nd half of atomic swap
	load upper immed.	lui $s1,20	$s1 = 20 * 2^{16}	Loads constant in upper 16 bits
Logical	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	nor $s1,$s2,$s3	$s1 = ~ ($s2 | $s3)	Three reg. operands; bit-by-bit NOR
	and immediate	andi $s1,$s2,20	$s1 = $s2 & 20	Bit-by-bit AND reg with constant
	or immediate	ori $s1,$s2,20	$s1 = $s2 | 20	Bit-by-bit OR reg with constant
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl $s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	beq $s1,$s2,25	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne $s1,$s2,25	if ($s1!= $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; for beq, bne
	set on less than unsigned	sltu $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than unsigned
	set less than immediate	slti $s1,$s2,20	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant
	set less than immediate unsigned	sltiu $s1,$s2,20	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant unsigned
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr $ra	go to $ra	For switch, procedure return
	jump and link	jal 2500	$ra = PC + 4; go to 10000	For procedure call